

# Testing Rollback

A tag pointed at an older commit, pushed once — and the running stack genuinely went backward, with no rebuild and no manual intervention.

---

tag: v0.1.0-rollback → commit 811af77    2026-08-16

## 00 What rollback actually is

Not a feature. Not a separate code path. `deploy.yml` has exactly one job, and it has no idea whether a given tag is a "release" or a "rollback" — that distinction exists only in the human's head, in which commit they chose to point the tag at. A rollback tag pointed backward runs through the identical resolve → pull → up sequence as any other deploy. The only reason this works without rebuilding anything is doc 3's design choice from the very start of the day: every push builds and tags by commit SHA, whether or not that commit ever becomes a release — so an old SHA's images are usually still sitting in the registry, untouched, whenever you need them again.

## 01 What was actually tested

Currently deployed: `v0.1.0`, resolving to commit `9992ba4`. Rollback target chosen deliberately: `811af77` — doc 9's confirmed fully-successful build, where all five images actually finished pushing (unlike `60bac55`, where only one image was ever manually pushed as a diagnostic, not the full set). Real rollback needs a real, complete prior version to land on.

```
git tag v0.1.0-rollback 811af77
git push origin v0.1.0-rollback
```

## 02 Which components are involved

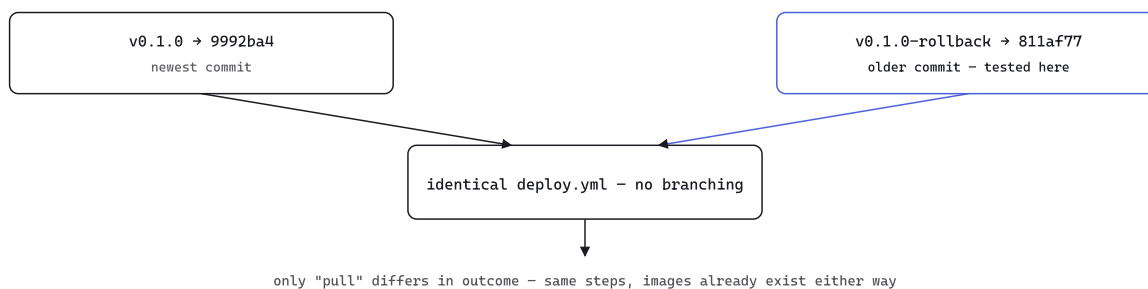
| COMPONENT                    | ROLE IN A ROLLBACK                                                                                                     |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| GitHub (router + controller) | Matches the tag push, resolves <code>deploy.yml</code> into a job — no awareness this is "a rollback"                  |
| Runner, on clubuntu          | Picks up the job, resolves the tag to a commit SHA, sets <code>IMAGE_TAG</code>                                        |
| Registry                     | Already holds the old SHA's images from when they were originally built — only ever <i>read</i> here, never written to |
| Docker Compose               | Diff's the new image references against what's running; recreates only the containers whose image actually changed     |
| Build pipeline               | Not involved at all — this is the whole point                                                                          |

### 03 What actually changed, container by container

| CONTAINER                  | BEFORE                          | AFTER                           | RECREATED?               |
|----------------------------|---------------------------------|---------------------------------|--------------------------|
| generic-mcp-gateway        | <code>gateway:9992ba4</code>    | <code>gateway:811af77</code>    | Yes                      |
| coder-commands-mcp         | <code>... :9992ba4</code>       | <code>... :811af77</code>       | Yes                      |
| codercommands-docker-mcp   | <code>docker-mcp:9992ba4</code> | <code>docker-mcp:811af77</code> | Yes                      |
| codercommands-postgres-mcp | <code>... :9992ba4</code>       | <code>... :811af77</code>       | Yes                      |
| codercommands-sync         | <code>sync:9992ba4</code>       | <code>sync:811af77</code>       | Yes                      |
| codercommands-postgres     | <code>postgres:17</code>        | <code>postgres:17</code>        | No — image never changed |

`postgres` staying untouched (still showing its original uptime, not recreated) is Compose's own diffing at work, not a special case written for this project — it only restarts what actually needs restarting.

### 04 The mechanism



Same picture as doc 4's original plan, and the FAQ's answer to "what happens during a rollback tag" — now backed by two real, confirmed deploys instead of description alone.

## 05 Verified

```
$ curl http://clubuntu.dala-cirius.ts.net:8100/healthz  
{"status":"ok"}
```

Reached directly over Tailscale from the dev laptop this time, not just from inside clubuntu — confirming the gateway is genuinely reachable across the tailnet on its published port, not only reachable locally.

## 06 What this proves

- ▶ Rollback cost one pull of already-built images — no rebuild, no wait for a fresh `docker buildx bake`
- ▶ Compose only recreated the five containers whose image reference actually changed, left `postgres` running untouched
- ▶ The exact same `deploy.yml`, unmodified, correctly served both a forward release and a backward rollback — because it was never told the difference